Chapter 3: Pandas: Data Wrangling

<{http://www.bookeducator.com/Textbook}learningobjective >Students will be able to iterate through DataFrames using appropriate methods (.itertuples(), .iterrows(), .items()) and determine when iteration versus vectorized operations is preferred
<{http://www.bookeducator.com/Textbook}learningobjective >Students will be able to apply vectorized calculations across entire columns using mathematical operators and built-in Pandas functions
<{http://www.bookeducator.com/Textbook}learningobjective >Students will be able to transform and recode categorical variables using vectorized methods (.map(), .replace()) instead of iterative approaches
<{http://www.bookeducator.com/Textbook}learningobjective >Students will be able to create derived features through vectorized operations combining multiple columns

3.1Introduction

Now that you have learned the core mechanics of Pandas DataFrames, this chapter shifts your focus from basic structure and access to **data wrangling**: the process of transforming, cleaning, and reshaping data so it can be analyzed, modeled, or deployed effectively.

A central theme of this chapter is understanding **how** Pandas is designed to be used. In particular, you will learn two fundamentally different ways of working with DataFrame values: iterating through data one row or cell at a time, and applying vectorized operations that act on entire columns at once.

Iteration is intuitive and often resembles how you might reason through a problem step by step. For that reason, it is an important learning bridge. However, iteration is rarely the preferred approach in professional Pandas workflows because it is slower, harder to maintain, and more error-prone than vectorized alternatives.

Vectorized operations, by contrast, leverage Pandas' internal optimizations to operate on entire columns efficiently and safely. Developing a column-oriented mindset is essential for writing scalable, readable, and performant data science code. This chapter begins that transition.

Specifically, this chapter will introduce techniques to:

- Iterate through DataFrames, including:
  - Columns
  - Rows
  - Individual cells
- Apply vectorized (column-level) calculations and transformations
- Relabel, recode, and transform values efficiently
- Prepare data for downstream tasks such as modeling, visualization, and machine learning pipelines

The techniques in this chapter form the foundation for most real-world data workflows. While iteration still has specific and valid use cases, you should begin thinking of vectorized operations as the default approach whenever possible.

3.2Iterating through DataFrames

One of the most common tasks associated with Pandas DataFrames is iterating through their contents. While iteration is not the preferred approach for most analytical tasks, it remains important for understanding how DataFrames are structured and for solving problems where each row depends on the result of a previous row.

In Pandas, iteration can occur across columns, rows, or values. Each approach has different performance characteristics, levels of flexibility, and appropriate use cases.

Iterating through Columns

By default, iterating directly over a DataFrame yields the column labels, not the column values themselves:

```
import pandas as pd

df = pd.DataFrame(
  {
    &quot;age&quot;:[29, 55, 65, 18],
    &quot;gender&quot;:[&quot;male&quot;, &quot;female&quot;, &quot;male&quot;,
&quot;female&quot;],
    &quot;hr1&quot;:[98.1, 78, 65, 64],
    &quot;hr2&quot;:[110, 120, 129, 141],
    &quot;hr3&quot;:[76, 87, 77, 59]
  },
  index=[&quot;p1&quot;, &quot;p2&quot;, &quot;p3&quot;, &quot;p4&quot;]
)

for col in df:
  print(col)


# Output:
# age
# gender
# hr1
# hr2
# hr3
```

Each value produced by the loop is a column label. These labels can then be used to retrieve the full column as a Series:

```
for col in df:
  print(df[col])
  print()


# Output:
# p1    29
# p2    55
# p3    65
# p4    18
# Name: age, dtype: int64
#
# p1      male
# p2    female
# p3      male
# p4    female
# Name: gender, dtype: object
#
# p1    98.1
# p2    78.0
# p3    65.0
# p4    64.0
# Name: hr1, dtype: float64
#
# p1    110
# p2    120
# p3    129
# p4    141
# Name: hr2, dtype: int64
#
# p1    76
# p2    87
# p3    77
# p4    59
# Name: hr3, dtype: int64
```

Conceptually, this loop is equivalent to manually accessing each column by name. This form of iteration is rarely needed in practice, but it helps reinforce that a DataFrame is fundamentally a collection of labeled columns.

```
print(df[&quot;age&quot;])
print(df[&quot;gender&quot;])
print(df[&quot;hr1&quot;])
print(df[&quot;hr2&quot;])
print(df[&quot;hr3&quot;])
```

```
# Output:
# p1    29
# p2    55
# p3    65
# p4    18
# Name: age, dtype: int64
# p1      male
# p2    female
# p3      male
# p4    female
# Name: gender, dtype: object
# p1    98.1
# p2    78.0
# p3    65.0
# p4    64.0
# Name: hr1, dtype: float64
# p1    110
# p2    120
# p3    129
# p4    141
# Name: hr2, dtype: int64
# p1    76
# p2    87
# p3    77
# p4    59
# Name: hr3, dtype: int64
```

Iterating through Rows

Row-based iteration is more common than column iteration, especially when calculations depend on prior rows. Pandas provides several methods that trade off speed and flexibility:

- **itertuples()** – fastest, read-only access
- **iterrows()** – slower, more flexible access

itertuples() returns each row as an immutable tuple-like object. This is the fastest way to iterate through rows when you only need to read values:

```
for row in df.itertuples():
  print(row)
```

```
# Output
# Pandas(Index='p1', age=29, gender='male', hr1=98.1, hr2=110, hr3=76)
# Pandas(Index='p2', age=55, gender='female', hr1=78.0, hr2=120, hr3=87)
# Pandas(Index='p3', age=65, gender='male', hr1=65.0, hr2=129, hr3=77)
# Pandas(Index='p4', age=18, gender='female', hr1=64.0, hr2=141, hr3=59)
```

Values can be accessed using attribute-style (row.age) or positional access, but not dictionary-style indexing:

```
for row in df.itertuples():
  print(row.Index, row.age, row.gender, row.hr1, row.hr2)


# Output:
# p1 29 male 98.1 110
# p2 55 female 78.0 120
# p3 65 male 65.0 129
# p4 18 female 64.0 141
```

Because itertuples() returns a tuple-like object, dictionary-style indexing (row['age']) does not work:

```
for row in df.itertuples():
  print(row.Index, '\t', end='')
  print(row['age'], '\t', end='')
  print(row['gender'], '\t', end='')
  print(row['hr1'], '\t', end='')
  print(row['hr2'])


# Output:
# p1
# ---------------------------------------------------------------------------
# TypeError                                 Traceback (most recent call last)
# &lt;ipython-input-60-96762764c160&gt; in &lt;cell line: 1&gt;()
#       1 for row in df.itertuples():
#       2   print(row.Index, '\t', end='')
# ----&gt; 3   print(row['age'], '\t', end='')
#       4   print(row['gender'], '\t', end='')
#       5   print(row['hr1'], '\t', end='')
# TypeError: tuple indices must be integers or slices, not str
```

Rows returned by itertuples() are immutable, so direct assignment is not supported:

```
for row in df.itertuples():
  row.Index = &quot;p5&quot;


# Output:
# ---------------------------------------------------------------------------
# TypeError                                 Traceback (most recent call last)
# &lt;ipython-input-10-88c49dc58294&gt; in &lt;module&gt;()
#       1 for row in df.itertuples():
# ----&gt; 2   row.Index = 'p5'
#
# TypeError: 'Pandas' object does not support item assignment
```

iterrows() returns each row as a Pandas Series along with its index label. This makes it easier to use dictionary-style access to values (row['age']), but it is slower than itertuples().

```
for index, row in df.iterrows():
  print(f&quot;Index: {index}\n{row}&quot;)
  print()


# Output:
# Index: p1
# age         29
# gender    male
# hr1       98.1
# hr2        110
# hr3         76
# Name: p1, dtype: object
#
# Index: p2
# age           55
# gender    female
# hr1         78.0
# hr2          120
# hr3           87
# Name: p2, dtype: object
```

Although iterrows() is more flexible, it is slower than itertuples() and should be avoided for large datasets. In all cases, both approaches are significantly slower than vectorized operations, which we will cover next.

If you need to update values while iterating, a common pattern is to iterate with iterrows() and write changes back to the original DataFrame using .at[] or .loc[].

Iterating through Cells

items() iterates over columns as (column_name, Series) pairs. This is useful when you want access to both the column label and all values in that column.

```
for key, value in df.items():
  print(key)
  print(value)
  print()


# Output
# age
# p1    29
# p2    55
# p3    65
# p4    18
# Name: age, dtype: int64
#
# gender
# p1      male
# p2    female
# p3      male
# p4    female
# Name: gender, dtype: object
#
# hr1
# p1    98.1
# p2    78.0
# p3    65.0
# p4    64.0
# Name: hr1, dtype: float64
#
# hr2
# p1    110
# p2    120
# p3    129
# p4    141
# Name: hr2, dtype: int64
#
# hr3
# p1    76
# p2    87
# p3    77
# p4    59
```

```
# Name: hr3, dtype: int64
```

Because this approach separates the column label (key) from the values (a Series), you can nest loops and apply additional logic while still keeping your code readable.

```
import pandas as pd

for key, values in df.items():
  for value in values:
    print(value, end=&quot;&quot;)

    if pd.api.types.is_number(value):
      print(&quot;\tnumeric&quot;, end=&quot;&quot;)

      if value &gt;= 80:
        print(&quot;\tflagged&quot;)
      else:
        print()
    else:
      print()


# Output:
# 29    numeric
# 55    numeric
# 65    numeric
# 18    numeric
# male
# female
# male
# female

# 98.1  numeric flagged

# 78.0  numeric

# 65.0  numeric

# 64.0  numeric
# 110   numeric flagged
# 120   numeric flagged
# 129   numeric flagged
# 141   numeric flagged
# 76    numeric
# 87    numeric flagged
# 77    numeric
# 59    numeric
```

Practice: Why Iterate At All?

At this point, you may be wondering why iteration is ever necessary. While vectorized operations are preferred, some problems involve row-level dependencies where each calculation relies on the result of a prior row.

This example models a vaccine inventory where each day's beginning inventory depends on the ending inventory from the previous day, making iteration appropriate.

In the code below, we will use the random number generator provided in the Numpy package to generate 10 random shipment and demand quantities and place them in a DataFrame called df_vax:

```
import numpy as np
import pandas as pd

days = pd.date_range(start=&quot;2024-01-01&quot;, periods=10, freq=&quot;D&quot;)
shipments = np.random.randint(50, 200, size=len(days))
administered = np.random.randint(100, 150, size=len(days))

df_vax = pd.DataFrame({
  &quot;date&quot;: days,
  &quot;shipments&quot;: shipments,
  &quot;administered&quot;: administered
})

print(df_vax)


# Output:
# 0 2024-01-01         67           115
# 1 2024-01-02        128           139
# 2 2024-01-03        146           111
# ...
```

See if you can complete this task on your own before examining the code below:

```
df_vax[&quot;beginning_inventory&quot;] = 0
df_vax[&quot;ending_inventory&quot;] = 0
inventory = 100
```

```
        for index, row in df_vax.iterrows():
          df_vax.at[index, &quot;beginning_inventory&quot;] = inventory
          inventory = inventory + row[&quot;shipments&quot;] -
row[&quot;administered&quot;]
          df_vax.at[index, &quot;ending_inventory&quot;] = inventory

        print(df_vax)


        # Output:
        #         date  shipments  administered  beginning_inventory  ending_inventory
        # 0 2024-01-01         67           115                  100                 52
        # 1 2024-01-02        128           139                   52                 41
        # 2 2024-01-03        146           111                   41                 76
        # ...
```

This example demonstrates a legitimate use case for iteration: cumulative calculations where each row depends on previous results.

3.3Vectorized Calculations and Functions

In the prior section, you learned how to iterate through DataFrame rows. Iteration is necessary when there is a true row dependency, such as when the ending inventory of one row becomes the beginning inventory of the next row. However, when each row can be computed independently, you should prefer vectorized calculations and functions (column-level operations). Vectorized operations are usually much faster because Pandas and NumPy perform the looping in optimized, low-level code instead of running a Python loop one row at a time.

A helpful rule of thumb is this: if your calculation can be written as “do the same thing to every value in a column,” it is probably a good fit for vectorization. If your calculation requires remembering or updating a value from the previous row, then you may need iteration.

Vectorized functions in Pandas operate on entire columns (or arrays) at once. Instead of running thousands of small Python steps, Pandas passes the work to efficient NumPy-based operations under the hood. Vectorized operations

are often faster than row-wise loops and can be easier to read for common tasks.

Copy the code below to create a basic DataFrame similar to those used earlier:

```
import pandas as pd

df = pd.DataFrame(
  {
    &quot;age&quot;: [29, 55, 65, 18],
    &quot;gender&quot;: [&quot;male&quot;, &quot;female&quot;, &quot;male&quot;,
&quot;female&quot;],
    &quot;hr1&quot;: [98.1, 78.0, 65.0, 64.0],
    &quot;hr2&quot;: [110, 120, 129, 141],
    &quot;hr3&quot;: [76, 87, 77, 59],
  },
  index=[&quot;p1&quot;, &quot;p2&quot;, &quot;p3&quot;, &quot;p4&quot;],
)

print(df)


# Output:
#     age  gender   hr1  hr2  hr3
# p1   29    male  98.1  110   76
# p2   55  female  78.0  120   87
# p3   65    male  65.0  129   77
# p4   18  female  64.0  141   59
```

To calculate the average heart rate (hr_mean) for each person using a vectorized operation, select the heart rate columns as a group and then use a built-in function:

```
hr_cols = [&quot;hr1&quot;, &quot;hr2&quot;, &quot;hr3&quot;]
df[&quot;hr_mean&quot;] = df[hr_cols].mean(axis=1)

print(df)


# Output:
#     age  gender   hr1  hr2  hr3    hr_mean
# p1   29    male  98.1  110   76  94.700000
# p2   55  female  78.0  120   87  95.000000
# p3   65    male  65.0  129   77  90.333333
```

```
# p4   18  female  64.0  141   59  88.000000
```

We call this a vectorized calculation or operation a calculation that performs an element-wise operation on an entire column (or arrays) without explicitly looping over each row in Python because the column is processed as a whole. This approach is usually much faster than writing a Python loop, especially as the dataset grows, because the looping happens in optimized internal code rather than in Python.

Vectorized operations can also be used to create transformed features. For example, if you want a square-root transformation of age, you can create a new column in one line. Using NumPy for common math functions is a clear, modern style that students will see often:

```
import numpy as np

df[&quot;age_sqrt&quot;] = np.sqrt(df[&quot;age&quot;])

print(df)


# Output (table will include the new column):
#     age  gender   hr1  hr2  hr3    hr_mean  age_sqrt
# p1   29    male  98.1  110   76  94.700000  5.385165
# p2   55  female  78.0  120   87  95.000000  7.416198
# p3   65    male  65.0  129   77  90.333333  8.062258
# p4   18  female  64.0  141   59  88.000000  4.242641
```

You can also use mathematical functions from NumPy along with built-in Pandas functions. For example, Pandas can compute skewness for a Series, so we can compare transformations of hr1 and decide which transformation moves skewness closer to zero:

```
import numpy as np

print(df[&quot;hr1&quot;].skew())
```

```
print(np.sqrt(df[&quot;hr1&quot;]).skew())
print(np.cbrt(df[&quot;hr1&quot;]).skew())
print(np.log(df[&quot;hr1&quot;]).skew())
df[&quot;hr1_ln&quot;] = np.log(df[&quot;hr1&quot;])
print(df)


# Output:

# 1.183647262060071

# 1.095475282166147

# 1.0657163814570545

# 1.005928426738866
```

	age	gender	hr1	hr2	hr3	hr_mean	age_sqrt	hr1_ln
p1	29	male	98.1	110	76	94.700000	5.385165	4.585987
p2	55	female	78.0	120	87	95.000000	7.416198	4.356709
p3	65	male	65.0	129	77	90.333333	8.062258	4.174387
p4	18	female	64.0	141	59	88.000000	4.242641	4.158883

There is one more technique that is sometimes useful: **.apply()**. The *.apply()* method can help when you have custom logic that does not fit cleanly into built-in vectorized steps. However, it still calls your Python function many times, so it is usually slower than built-in vectorized operations. A good strategy is to try a vectorized approach first and use *.apply()* when a clean vectorized solution is not available.

For example, suppose we want to calculate heart rate variability (HRV) for each person. HRV here is the difference between the maximum and minimum heart rates across hr1, hr2, and hr3. This is a good case for a vectorized solution:

```
hr_cols = [&quot;hr1&quot;, &quot;hr2&quot;, &quot;hr3&quot;]
df[&quot;hrv&quot;] = df[hr_cols].max(axis=1) - df[hr_cols].min(axis=1)

print(df)


# Output:
#     age  gender   hr1  hr2  hr3    hr_mean  age_sqrt    hr1_ln   hrv
# p1   29    male  98.1  110   76  94.700000  5.385165  4.585987  34.0
# p2   55  female  78.0  120   87  95.000000  7.416198  4.356709  42.0
# p3   65    male  65.0  129   77  90.333333  8.062258  4.174387  64.0
# p4   18  female  64.0  141   59  88.000000  4.242641  4.158883  82.0
```

If you cannot express your custom logic using built-in functions or simple vectorized steps, then *.apply()* is a reasonable backup option. The example below produces the same HRV values, but it is typically slower than the vectorized approach above:

```
def calculate_hrv(row):
  heart_rates = [row[&quot;hr1&quot;], row[&quot;hr2&quot;], row[&quot;hr3&quot;]]

  return max(heart_rates) - min(heart_rates)

df[&quot;hrv_apply&quot;] = df.apply(calculate_hrv, axis=1)
print(df[[&quot;hrv&quot;, &quot;hrv_apply&quot;]])


# Output:
#      hrv  hrv_apply
# p1  34.0       34.0
# p2  42.0       42.0
# p3  64.0       64.0
# p4  82.0       82.0
```

The table below summarizes the advantages, disadvantages, and best use cases of each method of DataFrame manipulation covered so far:

Table 3.1

Feature	Row-by-Row Iteration	Vectorized Operations with Math Operators	Vectorized Functions (e.g., .mean())	.apply() Function

Feature	Row-by-Row Iteration	Vectorized Operations with Math Operators	Vectorized Functions (e.g., .mean())	.apply() Function
Description	Iterate through rows using a loop (e.g., .iterrows()).	Operate on entire columns using mathematical operators (e.g., +, -, /).	Apply predefined Pandas/NumPy functions to columns (e.g., .mean(), .sum()).	Apply a custom function row-wise or column-wise.
Speed	**Slowest**: loops in Python and adds overhead for each row.	**Fast**: uses optimized internal operations for element-wise math.	**Fast**: uses optimized implementations for common statistics and aggregations.	**Moderate**: often slower than built-in vectorized operations because it repeatedly calls a Python function.
Advantages	Useful when row dependencies exist and when you need step-by-step logic.	Simple and very efficient for basic arithmetic across entire columns.	Efficient for aggregation with minimal code.	Flexible for custom logic when a clean vectorized solution is not available.
Use Case	When a row depends on previous rows and values must be updated iteratively.	Element-wise calculations across a column or selected columns.	Common statistics such as mean, sum, min, max, and standard deviation.	Custom logic that cannot be expressed cleanly using built-in vectorized steps.

3.4Relabel Values

Relabeling values is a common task in data analytics, especially when preparing data for machine learning models that cannot directly use categorical (text) values. For example, we might encode gender as 0 for female and 1 for male. Pandas offers multiple ways to relabel values, ranging from iterative methods (flexible but slower) to fast, vectorized methods (preferred when possible). The following sections cover these approaches.

Iterative Approach

One way to relabel values is by iterating through rows with *.itertuples()*, which creates a named tuple for each row. When you use *.itertuples()*, the clearest and safest way to access a column is by attribute name (for example, *row.gender*) rather than by numeric position (for example, *row[2]*). Numeric positions are easy to get wrong because they depend on the current column order and also include the row index.

```
for row in df.itertuples():
  if row.gender == &quot;female&quot;:
    print(0)
  else:
    print(1)


# Output:
# 1
# 0
# 1
# 0
```

To update values, you can iterate and assign into the DataFrame using *.at* (label-based scalar access). This is usually faster and more direct than

updating with *.loc* inside a loop. In practice, if you are already looping, prefer scalar setters (*.at* or *.iat*).

```
df_new = df.copy()

for row in df_new.itertuples():
  df_new.at[row.Index, &quot;gender&quot;] = 0 if row.gender == &quot;female&quot;
else 1

df_new


# Output:
#     age gender   hr1  hr2  hr3  age_sqrt    hr1_ln   hrv  hrv_apply
# p1   29      1  98.1  110   76  5.385165  4.585987  34.0       34.0
# p2   55      0  78.0  120   87  7.416198  4.356709  42.0       42.0
# p3   65      1  65.0  129   77  8.062258  4.174387  64.0       64.0
# p4   18      0  64.0  141   59  4.242641  4.158883  82.0       82.0
```

You can also combine *enumerate()* with *.itertuples()* to track the row position (0, 1, 2, ...). This is useful if you want to update values using *.iloc* or *.iat*, which are position-based. However, remember that position-based updates require the correct column position, which can change if column order changes.

```
for i, row in enumerate(df.itertuples()):
  if row.gender == &quot;female&quot;:
    print(i, 0)
  else:
    print(i, 1)


# Output:
# 0 1
# 1 0
# 2 1
# 3 0
```

Update Values Using .iloc or .iat

If you want to update using *.iat*, first compute the numeric column index for the column you want to update. This avoids “magic numbers” (like assuming gender is always column 1) and makes your code more resilient if the DataFrame changes.

```
df_new = df.copy()
gender_col = df_new.columns.get_loc(&quot;gender&quot;)

for i, row in enumerate(df_new.itertuples()):
  df_new.iat[i, gender_col] = 0 if row.gender == &quot;female&quot; else 1

print(df_new)


# Output:
#     age  gender   hr1  hr2  hr3  age_sqrt    hr1_ln   hrv  hrv_apply
# p1   29       1  98.1  110   76  5.385165  4.585987  34.0       34.0
# p2   55       0  78.0  120   87  7.416198  4.356709  42.0       42.0
# p3   65       1  65.0  129   77  8.062258  4.174387  64.0       64.0
# p4   18       0  64.0  141   59  4.242641  4.158883  82.0       82.0
```

Vectorized Relabeling

For standard relabeling tasks, vectorized approaches are usually the best option because they are both fast and easy to read. A good rule of thumb is: use *.map()* for simple dictionary recoding on one column, use *.replace()* when you want broader replacement behavior (possibly across multiple values or columns), and use *np.where()* when the new value depends on a condition.

Vectorized Relabeling with .map()

.map() is ideal when you have a clear “lookup table” (dictionary) for how to convert one set of labels into another.

```
df_map = df.copy()
gender_map = {&quot;female&quot;: 0, &quot;male&quot;: 1}
df_map[&quot;gender&quot;] = df_map[&quot;gender&quot;].map(gender_map)

print(df_map)


# Output:
#     age  gender   hr1  hr2  hr3  age_sqrt    hr1_ln   hrv  hrv_apply
# p1   29       1  98.1  110   76  5.385165  4.585987  34.0       34.0
# p2   55       0  78.0  120   87  7.416198  4.356709  42.0       42.0
# p3   65       1  65.0  129   77  8.062258  4.174387  64.0       64.0
# p4   18       0  64.0  141   59  4.242641  4.158883  82.0       82.0
```

If a value is not found in your mapping dictionary, *.map()* returns *NaN* for that row. This is useful because it helps you detect unexpected or “dirty” categories, but it also means you may need to handle missing values before converting to an integer type.

Vectorized Relabeling with .replace()

.replace() is similar to *.map()*, but it is designed around “find-and-replace” behavior and can be convenient when you want to replace multiple old values at once. It can also be applied across multiple columns when needed.

```
# This is necessary (only for now) because Pandas is updating their library
pd.set_option(&quot;future.no_silent_downcasting&quot;, True)
df_replace = df.copy()

df_replace[&quot;gender&quot;] = (
  df_replace[&quot;gender&quot;]
    .replace({&quot;female&quot;: 0, &quot;male&quot;: 1})
    .astype(&quot;int64&quot;)
)

print(df_replace)


# Output:
#     age  gender   hr1  hr2  hr3  age_sqrt    hr1_ln   hrv  hrv_apply
# p1   29       1  98.1  110   76  5.385165  4.585987  34.0       34.0
# p2   55       0  78.0  120   87  7.416198  4.356709  42.0       42.0
# p3   65       1  65.0  129   77  8.062258  4.174387  64.0       64.0
# p4   18       0  64.0  141   59  4.242641  4.158883  82.0       82.0
```

Vectorized Relabeling with np.where()

np.where() is ideal when the recoding is based on a condition. Think of it as a vectorized “if/else” for an entire column.

```
import numpy as np

df_where = df.copy()
df_where[&quot;gender&quot;] = np.where(df_where[&quot;gender&quot;] ==
&quot;female&quot;, 0, 1)

print(df_where)


# Output:
#     age  gender   hr1  hr2  hr3  age_sqrt    hr1_ln   hrv  hrv_apply
# p1   29       1  98.1  110   76  5.385165  4.585987  34.0       34.0
# p2   55       0  78.0  120   87  7.416198  4.356709  42.0       42.0
# p3   65       1  65.0  129   77  8.062258  4.174387  64.0       64.0
# p4   18       0  64.0  141   59  4.242641  4.158883  82.0       82.0
```

Vectorized Relabeling with Filtered Assignment Statement

Just so you're aware when you see it, relabelling can also be accomplished with a basic DataFrame filter and assignment statement if there is just one simple change to make.

```
df_new = df.copy()
df_new.loc[df_new['gender'] == 'female', 'gender'] = 0
df_new.loc[df_new['gender'] == 'male', 'gender'] = 1

print(df_new)


# Output:
#     age  gender   hr1  hr2  hr3  age_sqrt    hr1_ln   hrv  hrv_apply
# p1   29       1  98.1  110   76  5.385165  4.585987  34.0       34.0
# p2   55       0  78.0  120   87  7.416198  4.356709  42.0       42.0
# p3   65       1  65.0  129   77  8.062258  4.174387  64.0       64.0
# p4   18       0  64.0  141   59  4.242641  4.158883  82.0       82.0
```

Ensuring Correct Data Types

After relabeling, always check your column data types. Many machine learning workflows require numeric columns to be stored as numeric types (not as generic objects).

```
df_new.dtypes


# Output:
# age           int64
# gender       object
# hr1         float64
# hr2           int64
# hr3           int64
# hr_mean     float64
# age_sqrt    float64
# hr1_ln      float64
# dtype: object
```

Even though gender was relabeled as numbers, the type may still be *object* if the column previously contained text or mixed values. If you are confident there are no missing values, cast it to *int64*. If missing values are possible, consider using Pandas' nullable integer type (*Int64*) instead.

```
df_new[&quot;gender&quot;] = df_new[&quot;gender&quot;].astype(&quot;int64&quot;)
df_new.dtypes


# Output:
# age           int64
# gender        int64
# hr1         float64
# hr2           int64
# hr3           int64
# hr_mean     float64
# age_sqrt    float64
# hr1_ln      float64
# dtype: object
```

All good.

Summary of Relabeling Techniques

Table 3.2

Technique	Method	Pros	Cons
Iterative	.itertuples() + .at	Flexible for complex logic and row-by-row side effects	Slower for large datasets; more code to maintain
Iterative with Position	enumerate() + .iat	Fast scalar updates when you already know row/column positions	Requires correct column positions; can be fragile if columns move
Vectorized Lookup	.map()	Fast, readable dictionary recoding for a single Series	Unmapped categories become NaN; may require missing-value handling
Vectorized Replace	.replace()	Convenient find-and-replace style recoding; good for multiple replacements	Less explicit "unmapped" signal than .map(); can be overused when .map() is clearer
Vectorized Conditional	np.where()	Excellent for binary or threshold-based logic; reads like an if/else	Becomes harder to read with many categories (better to use .map() or .replace())
Data Type Casting	.astype()	Ensures correct data types for modeling and downstream analysis	May require nullable types if missing values exist

Key Takeaways

- Iterative methods can be useful for complex, row-dependent logic, but they are slower and should not be your default approach for relabeling.

- Vectorized methods such as *.map()*, *.replace()*, and *np.where()* are usually the best choice for standard recoding tasks because they are fast and readable.
- Always verify and (if needed) convert column data types after relabeling to ensure compatibility with downstream tasks.

3.5Working with Dates

Converting dates into a usable format is a common task in the data preparation phase of most data science projects. Most modeling algorithms can only work with numeric data. Therefore, dates must be converted into some other form. The two most common practices are:

- Convert to the number of years/months/days/hours/minutes/seconds until or since a given date/time
- Convert to the number of years/months/days/hours/minutes/seconds between two date/time columns

The examples below demonstrate both. Start by creating a simple DataFrame:

```
df['signup_date'] = ['02/14/2019', '01/05/2018', '05/23/2020', '12/10/2019']
df['complete_date'] = ['02/27/2020', '10/22/2020', '09/11/2021', '07/05/2022']
df
```

	age	gender	hr1	hr2	hr3	hr_mean	age_sqrt	hr1_ln	signup_date	complete_date
p1	29	male	98.1	110	76	94.700000	5.385165	4.585987	02/14/2019	02/27/2020
p2	55	female	78.0	120	87	95.000000	7.416198	4.356709	01/05/2018	10/22/2020
p3	65	male	65.0	129	77	90.333333	8.062258	4.174387	05/23/2020	09/11/2021
p4	18	female	64.0	141	59	88.000000	4.242641	4.158883	12/10/2019	07/05/2022

Notice, however, that just because we entered data in a date format, that does not mean Pandas automatically recognizes the data as a datetime format:

```
df.dtypes


# Output
# age                int64
# gender            object
# hr1              float64
# hr2                int64
# hr3                int64
# hr_mean          float64
# age_sqrt         float64
# hr1_ln           float64
# signup_date       object
# complete_date     object
# dtype: object
```

Converting to DateTime Data Type

Convert those two date columns to a recognized datetime format (note that the processes followed here for dates work similarly for times).

```
df['signup_date'] = pd.to_datetime(df['signup_date'])
df['complete_date'] = pd.to_datetime(df['complete_date'])

print(df.dtypes)


# Output
# age                       int64
# gender                   object
# hr1                     float64
# hr2                       int64
# hr3                       int64
# age_sqrt                float64
# hr1_ln                  float64
# hrv                     float64
# hrv_apply               float64
# signup_date      datetime64[ns]
# complete_date    datetime64[ns]
# dtype: object
```

Now the two dates are formatted as datetime64[ns] and notice that the dates were reformatted as yyyy-mm-dd:

```
df
```

	age	gender	hr1	hr2	hr3	hr_mean	age_sqrt	hr1_ln	signup_date	complete_date
p1	29	male	98.1	110	76	94.700000	5.385165	4.585987	2019-02-14	2020-02-27
p2	55	female	78.0	120	87	95.000000	7.416198	4.356709	2018-01-05	2020-10-22
p3	65	male	65.0	129	77	90.333333	8.062258	4.174387	2020-05-23	2021-09-11
p4	18	female	64.0	141	59	88.000000	4.242641	4.158883	2019-12-10	2022-07-05

Extracting Date and Time Components

Once a column is stored as a Pandas datetime type, we can easily extract meaningful components such as the year, month, day, or day of the week. These derived features are commonly used in exploratory analysis and machine learning models.

Pandas provides the **.dt** accessor, which allows vectorized access to date and time properties without iteration.

```
df[&quot;signup_year&quot;] = df[&quot;signup_date&quot;].dt.year
df[&quot;signup_month&quot;] = df[&quot;signup_date&quot;].dt.month
df[&quot;signup_day&quot;] = df[&quot;signup_date&quot;].dt.day
df[&quot;signup_day_of_week&quot;] = df[&quot;signup_date&quot;].dt.dayofweek
df
```

```
# Output:
#     signup_date  signup_year  signup_month  signup_day  signup_day_of_week
# p1  2019-02-14         2019             2          14                   3
# p2  2018-01-05         2018             1           5                   4
# p3  2020-05-23         2020             5          23                   5
# p4  2019-12-10         2019            12          10                   1
```

The **day_of_week** feature uses a numeric convention where Monday is 0 and Sunday is 6. This numeric encoding is convenient for modeling, but you can also extract readable labels if needed.

```
df[&quot;signup_day_name&quot;] = df[&quot;signup_date&quot;].dt.day_name()
print(df[['signup_date', 'signup_day_name']])


# Output:
#     signup_date signup_day_name
# p1  2019-02-14        Thursday
# p2  2018-01-05          Friday
# p3  2020-05-23        Saturday
# p4  2019-12-10         Tuesday
```

If a datetime column includes a time component, the same .dt accessor can extract hour, minute, and second values using **.dt.hour**, **.dt.minute**, and **.dt.second**. These features are especially common in event data and time-series analysis.

Calculating Useful Metrics

Now, with the dates in the proper format, we can perform basic operations such as calculating the difference between a given static date and the dates in a DataFrame column, measured in days, and then stored into a new column:

```
anchor = pd.Timestamp(&quot;2023-01-01&quot;)
df[&quot;days_since_signup&quot;] = (anchor -
df[&quot;signup_date&quot;]).dt.days.astype(&quot;int64&quot;)

print(df[[&quot;signup_date&quot;, &quot;days_since_signup&quot;]])


# Output:
#    signup_date  days_since_signup
# p1  2019-02-14               1417
# p2  2018-01-05               1822
# p3  2020-05-23                953
```

```
# p4  2019-12-10              1118
```

The new column returns an integer representing the number of days from each signup date until a fixed anchor date. To make this work, we 1) created a *pd.Timestamp* object for the anchor date, 2) subtracted the signup date column from it, 3) accessed the *.dt.days* property of the resulting timedelta, and 4) cast to *int64* so the column stores clean integers.

We can also create a new column to represent the difference between two date columns in days:

```
df['days_to_completion'] = (df['complete_date'] - df['signup_date']).dt.days
df['days_to_completion'] = df['days_to_completion'].astype(int)

print(df[['signup_date', 'complete_date', 'days_to_completion']])


# Output
#    signup_date complete_date  days_to_completion
# p1  2019-02-14    2020-02-27                 378
# p2  2018-01-05    2020-10-22                1021
# p3  2020-05-23    2021-09-11                 476
# p4  2019-12-10    2022-07-05                 938
```

In addition, we can get the difference between two dates in any other time measurement we want, including years, months, weeks, hours, seconds, or anything else with a basic operation on the new days column:

```
df['years'] = df['days_to_completion']/365
df['weeks'] = df['days_to_completion']/7
df['months'] = df['days_to_completion']/(365/12)

print(df[['signup_date', 'complete_date', 'days_to_completion', 'years', 'weeks',
'months']])


# Output
#    signup_date complete_date  days_to_completion     years       weeks  \
# p1  2019-02-14    2020-02-27                 378  1.035616   54.000000
# p2  2018-01-05    2020-10-22                1021  2.797260  145.857143
# p3  2020-05-23    2021-09-11                 476  1.304110   68.000000
```

```
# p4  2019-12-10    2022-07-05                938  2.569863  134.000000
#
#        months
# p1  12.427397
# p2  33.567123
# p3  15.649315
# p4  30.838356
```

Now that we have created usable numeric features based on the date columns, we can drop the original date columns (signup_date and complete_date) if desired.

3.6Practice

See how well you understand the material by working through the practice problems below. Complete each problem in order, because later problems build on work from earlier ones.

Practice 1: Student Mental Health
Academic programs have noticed increasing concern about student mental health. Administrators collected a dataset that includes whether each student has experienced depression, anxiety, and panic attacks, as well as whether they have sought treatment. Download the dataset, upload it to your Google Drive, and import it into a notebook cell as a Pandas DataFrame named *df*. Print the first five rows to examine the data.

Several cleaning steps are needed. First, there was a >Age: each student's recorded age is one year younger than it should be. Fix this by adding 1 to every value in the existing *Age* column. Then verify that the change worked.

1. Print the mean of the original *Age* column.
2. Add 1 to every value in the *Age* column (modify the existing column).
3. Print the mean of the updated *Age* column and confirm it is exactly 1.0 higher than before.

Practice 2: Relabel Values

To prepare for predictive modeling, convert selected categorical values into numeric codes. Relabel *Gender* so that Female = 0 and Male = 1. Relabel *MaritalStatus* so that Yes = 1 and No = 0. After relabeling, ensure both columns are stored as integers (not objects). Print the first five rows of the updated DataFrame to confirm your results.

Practice 3: Working with Dates and Times

Convert the *Timestamp* column to a Pandas datetime type so that you can compute time differences. Assume every student started the survey at exactly 12:00 PM on the same date shown in their timestamp. Create a new integer column named *MinutesToFinish* that stores the number of minutes from 12:00 PM until the student's timestamp.

Finally, create two additional columns from *Timestamp*: *Hour* (0–23) and *DayOfWeek* (0=Monday, 6=Sunday). Print the first five rows showing *Timestamp*, *MinutesToFinish*, *Hour*, and *DayOfWeek*.

Practice 4: Iterate

Identify first-year students who reported having a panic attack and who *did* seek treatment. To practice iteration, use a loop to check each row one at a time. Print the entire row for each student who meets all criteria.

As an extension, count how many students meet the criteria and print the final count.

Practice 5: Vectorized Calculation

This chapter emphasized that vectorized operations are usually faster and clearer than row-by-row loops when there is no row dependency. Create a

new column named *HasAnyIssue* that equals 1 if the student reported depression *or* anxiety *or* a panic attack, and equals 0 otherwise. Use a vectorized approach (no for-loops).

Then, print the proportion of students with *HasAnyIssue* = 1.

Practice 6: Choosing Iteration vs. Vectorization

This chapter emphasized that choosing the *right* technique matters just as much as getting the correct result. In this practice problem, you will solve the same task two different ways and compare the approaches.

Create a new column called *RiskFlag* using the following rule:

- If a student is under 25 years old *and* reported anxiety, set *RiskFlag* = 1.
- Otherwise, set *RiskFlag* = 0.

Complete this task twice:

1. First, use row-by-row iteration (such as *iterrows()* or *itertuples()*).
2. Second, use a fully vectorized approach with boolean logic.

After completing both versions, verify that the two resulting *RiskFlag* columns are identical. Then briefly reflect (in a comment) on which approach is clearer and which would scale better to large datasets.

3.7Assignment

Complete the assignment below.

This assessment can be taken online.